Signal Processing and Normalization

1. Centering the EMG Signal

Objective: Remove the DC offset to center the raw EMG signal around zero.

1. **Calculate the Mean (DC Offset):**
   - Compute the mean of the raw EMG signal over a fixed window size (e.g., 100 samples).
2. **Subtract the Mean:**
   - Subtract the computed mean from each sample of the raw EMG signal to center it around zero.

Implementation:

1. Create a function `centerSignal` that takes the raw EMG signal array, an array to store the centered signal, and the window size as inputs.
2. Inside the function, calculate the mean of the raw signal.
3. Subtract the mean from each sample to center the signal.

2. Filtering the EMG Signal

Objective: Remove unwanted low-frequency and high-frequency noise from the centered EMG signal.

1. **High-Pass Filter:**
   - Design a simple high-pass filter with a cutoff frequency around 20-30 Hz.
   - Apply the filter to the centered EMG signal.
2. **Low-Pass Filter:**
   - Design a simple low-pass filter with a cutoff frequency around 450-500 Hz.
   - Apply the filter to the high-pass filtered signal.

Implementation:

1. Create two functions, `highPassFilter` and `lowPassFilter`, that apply the respective filters.
2. The `highPassFilter` function should use the formula:
   - `FilteredSignal_high-pass[i] = alpha * (FilteredSignal_high-pass[i-1] + CenteredSignal[i] - CenteredSignal[i-1])`, where `alpha` is the filter coefficient.
3. The `lowPassFilter` function should use the formula:
   - `FilteredSignal_low-pass[i] = beta * CenteredSignal[i] + (1 - beta) * FilteredSignal_low-pass[i-1]`, where `beta` is the filter coefficient.
4. Create a function `filterSignal` that applies these filters to the centered signal.

3. Rectifying the EMG Signal

Objective: Convert all values of the filtered EMG signal to positive values to prepare for normalization.

Implementation:

1. Create a function `rectifySignal` that takes the filtered signal array and an array to store the rectified signal.
2. Inside the function, convert each sample to its absolute value.

4. Normalizing the EMG Signal

Objective: Scale the rectified EMG signal using the Maximum Voluntary Contraction (MVC) value obtained during calibration.

1. **Calibrate and Record MVC:**
    - Instruct the user to perform a maximum voluntary contraction.
    - Record the EMG signal during this period and find the maximum value.
2. **Normalize the Signal:**
    - Divide each sample of the rectified EMG signal by the MVC value.

Implementation:

1. Create a function `calculateMVC` that takes an array of calibration data and its size, and returns the maximum value.
2. Create a function `normalizeSignal` that takes the rectified signal array, an array to store the normalized signal, the MVC value, and the window size.
3. Inside the `normalizeSignal` function, divide each sample of the rectified signal by the MVC value.

5. Complete Processing Pipeline

Objective: Integrate all the steps to process the raw EMG signal from acquisition to normalization.

Implementation:

1. Create a function `processEMGSignal` that integrates the steps to center, filter, rectify, and normalize the raw EMG signal.
2. The function should use a buffer size (e.g., 100 samples) and arrays to store the raw, centered, filtered, rectified, and normalized signals.
3. Include a global variable for the MVC value, which should be set during calibration.